Event Storming — Saga da Ordem de Serviço (v1, rascunho)

Rascunho inicial pra refinar com o grupo (item da Semana 0 do `divisao-tarefas-fase4.md`). Cobre o caminho feliz e os caminhos de compensação aprovados na reunião de 15/10/2026.

Legenda (pra quem não usou Event Storming antes)

Elemento	O que é	Convenção
🟧 **Evento de domínio**	Um fato que já aconteceu. Sempre no passado.	`OrcamentoGerado`, `PagamentoRecusado`
🟦 **Comando**	Uma ação/intenção, no imperativo. Dispara um evento quando bem-sucedido.	`GerarOrcamento`, `ProcessarPagamento`
🟪 **Política**	Regra "quando [evento], então [comando]" — é o que conecta um evento ao próximo comando, atravessando os serviços.	"Quando `OrdemDeServicoAberta`, então `GerarOrcamento`"
🟨 **Agregado**	A entidade "dona" do evento, responsável por garantir suas regras de negócio.	`OrdemDeServico`, `Orcamento`, `Execucao`
🧍 **Ator**	Quem inicia uma ação externamente.	Cliente
🟥 **Sistema externo**	Fora dos nossos serviços.	Mercado Pago
🔴 **Hot spot**	Pergunta em aberto ou ponto de atrito a resolver com o grupo.	ver seção final

Linha do tempo — caminho feliz

#	🟦 Comando	🟨 Agregado / Serviço	🟧 Evento resultante	🟪 Política (reação)
1	`AbrirOrdemDeServico` (ator: Cliente)	`OrdemDeServico` / OS Service	`OrdemDeServicoAberta`	Quando `OrdemDeServicoAberta` → disparar `GerarOrcamento` (fila `gerar-orcamento`)
2	`GerarOrcamento`	`Orcamento` / Billing Service	`OrcamentoGerado`	Quando `OrcamentoGerado` → OS Service publica na fila `orcamento-respondido`; status da OS vira `AGUARDANDO_APROVACAO`

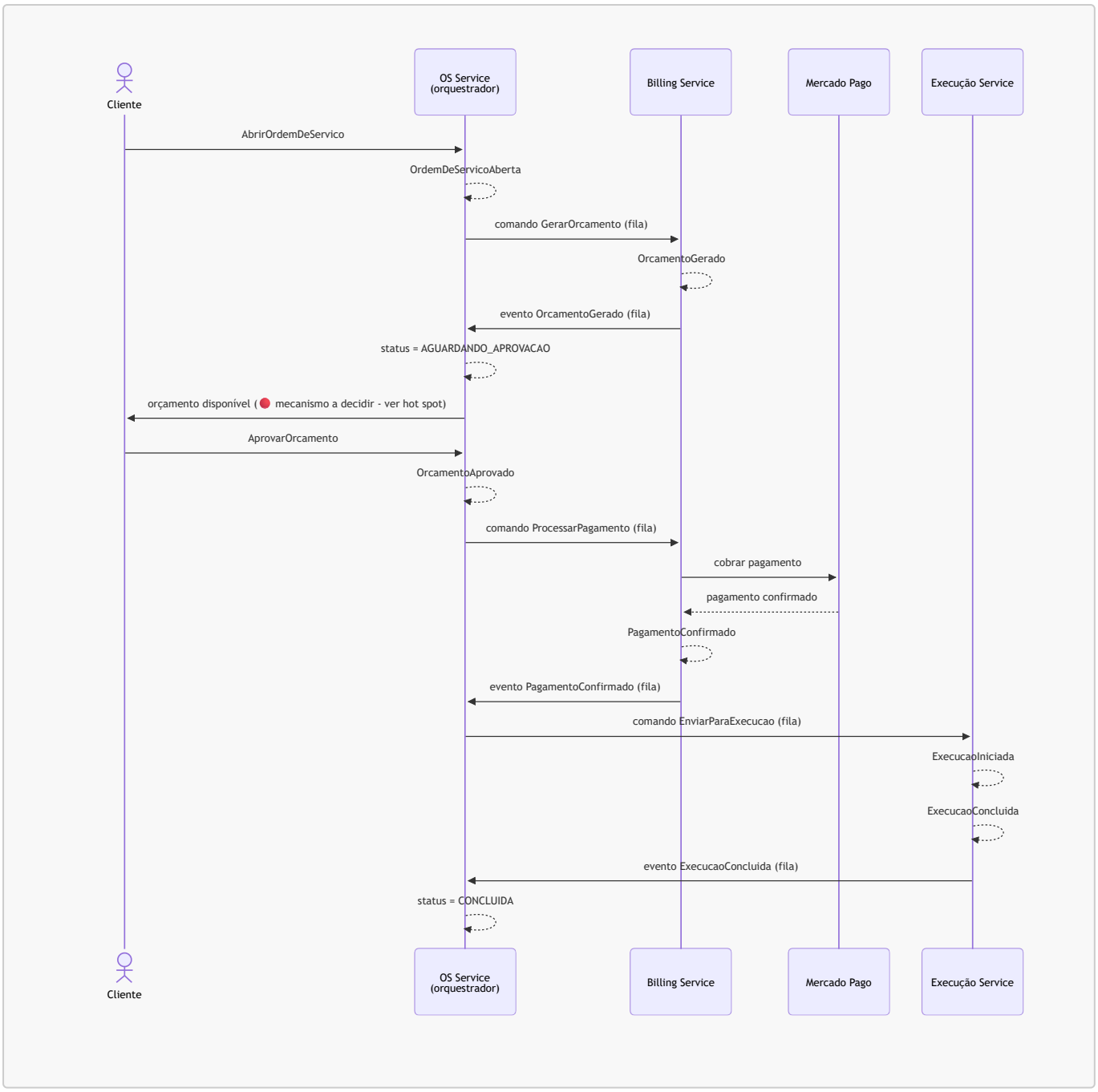
#	🟦 Comando	🟨 Agregado / Serviço	🟧 Evento resultante	🟪 Política (reação)
3	`AprovarOrcamento` (ator: Cliente)	`OrdemDeServico` / OS Service	`OrcamentoAprovado`	Quando `OrcamentoAprovado` → disparar `ProcessarPagamento` (Billing)
4	`ProcessarPagamento`	`Pagamento` / Billing Service (🟥 integra com Mercado Pago)	`PagamentoConfirmado`	Quando `PagamentoConfirmado` → disparar `EnviarParaExecucao` (fila `enviar-execucao`)
5	`EnviarParaExecucao`	`Execucao` / Execução Service	`ExecucaoIniciada`	—
6	`ConcluirExecucao`	`Execucao` / Execução Service	`ExecucaoConcluida`	Quando `ExecucaoConcluida` → OS Service publica na fila `execucao-concluida`; status da OS vira `CONCLUIDA`
7	*(implícito — entrega física)*	`OrdemDeServico` / OS Service	`OrdemDeServicoEntregue`	Fim do fluxo feliz

Linha do tempo — compensações (caminhos de falha)

Gatilho (🟧 evento de falha)	🟦 Comando de compensação	🟨 Agregado / Serviço	🟧 Evento resultante
`OrcamentoReprovado` (ator: Cliente reprova)	`CancelarOrdemDeServico`	`OrdemDeServico` / OS Service	`OrdemDeServicoCancelada`
`PagamentoRecusado` (Mercado Pago recusa)	`CancelarOrdemDeServico`	`OrdemDeServico` / OS Service	`OrdemDeServicoCancelada`
`ExecucaoFalhou` (ex.: falta de peça, fila indisponível)	`EstornarPagamento` **então** `CancelarOrdemDeServico`	`Pagamento` / Billing Service → `OrdemDeServico` / OS Service	`PagamentoEstornado` → `OrdemDeServicoCancelada`

> Nota da P7: a etapa de orçamento/pagamento no Billing Service deve aplicar Pessimistic View ou Reread Value antes de confirmar `PagamentoConfirmado`, pra evitar condição de corrida se o cliente aprovar enquanto outro processo mexe no mesmo orçamento.

Diagrama de sequência (caminho feliz)



Hot spots — perguntas em aberto pro grupo

1. **Contrato dos eventos:** schema exato do payload de cada mensagem SQS (campos obrigatórios, versionamento). Precisa ser combinado antes de cada dupla começar a implementar sua ponta — é o mesmo tipo de problema que travou a Fase 3 (contrato do JWT divergente entre Lambda e app).
2. **Consulta síncrona Billing → Execução:** o Billing precisa saber o preço dos itens de `Servico`/`ItemEstoque` (que agora vivem no Execução Service) pra gerar o orçamento. Isso é uma chamada REST síncrona na hora, ou os dados do catálogo vêm junto no comando inicial do OS Service?
3. **Onde mora o estado da Saga:** um campo de status na própria `OrdemDeServico`, ou uma tabela separada dedicada a acompanhar "em que passo está essa Saga" (mais rastreável, mas mais uma tabela pra manter)?
4. **Timeout de aprovação:** o que acontece se o cliente nunca aprovar/reprovar o orçamento? Precisa de um evento de expiração?
5. **Como o cliente sabe que o orçamento está pronto pra aprovar:** hoje não existe nenhum mecanismo de notificação — a aplicação sempre funcionou por consulta (`GET`

`/api/v1/ordens/cliente/{clienteId}`). Mantemos assim (cliente consulta quando quiser) ou adicionamos alguma notificação ativa (e-mail/push)? Não é exigido pelo enunciado da Fase 4, então é uma decisão de escopo, não só técnica.